Documentación Técnica Backend - Fichaday CTI

Sistema de Gestión de Presencia y Recursos Humanos Multi-Empresa

---

Índice

Documentación Técnica Backend - Fichaday CTI....1
Índice....1
Visión General del Proyecto....5
Características Principales....5
Arquitectura del Sistema....5
Stack Tecnológico....5
Estructura de Módulos (20 módulos)....6
Librerías y Dependencias....7
Dependencias de Producción....7
Dependencias de Desarrollo....9
Entidades de la Base de Datos....10
Resumen de Entidades (20 entidades)....10
Detalle Completo de Entidades (20 entidades)....12
Configuraciones del Sistema....24
Configuración Principal (main.ts)....24
Configuración de Módulos (app.module.ts)....24
Sistema de Seguridad....25
Guards (Protectores de Rutas)....25
JwtAuthGuard....25
RolesGuard....25
Decoradores Personalizados....26
@GetUser()....26
@Roles()....26
Estrategias de Passport....27

JwtStrategy ... 27
Interfaces ... 27
UserPayload ... 27
Contraseñas ... 28
Variables de Entorno ... 29
Archivo de Configuración del Backend (backend/.env) ... 29
Archivo de Configuración de Base de Datos (BBDD/.env) ... 30
Explicación Detallada de Variables ... 33
Variables de Base de Datos ... 33
Variables de Configuración de la App ... 33
Variables de Seguridad JWT ... 33
Variables de Lógica de Negocio ... 33
Variables de Generación de PIN ... 34
Variables de pgAdmin ... 34
Variables de Correo (SMTP Brevo) ... 34
Configuración Docker ... 34
Justificación del Uso de Docker ... 34
Proxy Nginx ... 35
Justificación del Uso de Nginx ... 35
Capa de Acceso: Nginx Proxy (Doble Dominio) ... 35
Comando de Despliegue ... 35
Conexión de Redes ... 37
Configuración Nginx Explicada ... 38
Servidor de Producción (fichaday.cticontrol.com) ... 38
Servidor de Test (fichadaytest.cticontrol.com) ... 39
Características de la Configuración ... 39
Base de Datos PostgreSQL ... 40
Justificación del Uso de PostgreSQL ... 40
Características Implementadas ... 40
pgAdmin ... 40
Propósito ... 40
Acceso ... 41
Conexión a Bases de Datos ... 41
Nota de Seguridad ... 41
Colección Postman ... 41
Directorio de Endpoints ... 41
Cómo Importar en Postman ... 41
Script de Despliegue Automático ... 42
Archivo: deploy.ps1 ... 42
Funcionalidades del Script ... 42
Ejecución del Script ... 42
Flujo del Script ... 42

Salida Esperada....43
Flujo del PIN....43
Descripción General....43
Fase 1: Generación del PIN (Bautismo)....43
Fase 2: Login del Trabajador (Día a Día)....44
Seguridad del PIN....45
Envío de Correo del PIN....45
Servicio de Correo (MailService)....45
Función: sendWorkerPin....45
Adjunto del Logo....45
Proveedor de Correo....46
Seed de Inicialización....46
Propósito del Seed....46
Servicio: SeedsService....46
Endpoint del Seed....46
Flujo de Inicialización Proyecto....47
Módulos del Sistema....47
1. Auth (Autenticación)....47
2. Users (Usuarios)....48
3. Company (Empresas)....48
4. Worker (Trabajadores)....49
5. Workday (Jornadas)....50
6. Schedule (Horarios)....50
7. TimeSlot (Franjas Horarias)....51
8. Attendance (Asistencia)....51
9. Absence (Ausencias)....52
10. Request (Solicitudes)....53
11. Issue (Incidencias)....53
12. Alarms (Alarmas)....54
13. Activity (Actividades)....54
14. Site (Obras/Sitios)....55
15. SiteLog (Fichajes en Obras)....55
16. IssueSite (Incidencias en Obras)....56
17. Mail (Correo)....56
18. Seed (Inicialización)....57
19. WorkerAuth (Autenticación PIN)....57
20. Common (Utilidades)....58
Flujo Completo de la Aplicación....58
Fase 1: Inicialización del Sistema....58
Fase 2: Configuración de Empresa....59
Fase 3: Planificación....59
Fase 4: Gestión de Personal....59

Fase 5: Operativa Diaria (Trabajador)..... 59
Fase 6: Gestión de Solicitudes..... 60
Fase 7: Gestión de Ausencias..... 60
Fase 8: Supervisión y Alarmas..... 61
Fase 9: Reportes y Estadísticas..... 61
Fase 10: Mantenimiento Automático..... 62
Guía de Despliegue..... 62
Despliegue Local (Desarrollo)..... 62
Requisitos Previos..... 62
Pasos de Despliegue..... 62
Despliegue en Producción (VPS)..... 64
Requisitos Previos..... 64
Pasos de Despliegue..... 64
Con el archivo env en la carpeta env..... 66
Resumen de Comandos del README..... 70
Instalación Local..... 70
Levantar Infraestructura Manualmente..... 70
Despliegue Producción (VPS)..... 71
Conclusión..... 71

Visión General del Proyecto

Fichaje CTI es un sistema completo de gestión de Recursos Humanos con arquitectura multi-tenant, desarrollado en **NestJS** + **TypeORM** + **PostgreSQL**. El sistema permite a múltiples empresas gestionar sus trabajadores, horarios, fichajes, solicitudes y ausencias de forma independiente y segura.

Características Principales

- **Multi-tenancy**: Aislamiento completo de datos por empresa
- **Seguridad JWT**: Autenticación basada en tokens con 3 roles (SUPERADMIN, ADMIN, WORKER)
- **Geolocalización**: Validación de fichajes por GPS con radio permitido
- **Horarios Rotativos**: Sistema avanzado de turnos con ciclos configurables
- **Sistema de PIN**: Login rápido para trabajadores con PIN de 6 dígitos
- **Notificaciones**: Alarmas en tiempo real con WebSocket
- **Limpieza Automática**: Cron jobs para mantenimiento de datos antiguos

---

Arquitectura del Sistema

Stack Tecnológico

- **Backend**: NestJS (Framework Node.js)
- **ORM**: TypeORM
- **Base de Datos**: PostgreSQL 15
- **Contenedores**: Docker + Docker Compose
- **Proxy**: Nginx 1.25 Alpine
- **Autenticación**: JWT + Passport
- **Validación**: class-validator + class-transformer

Estructura de Módulos (20 módulos)

backend/src/

├── **auth/** # Autenticación y autorización

├── **users/** # Gestión de usuarios

├── **company/** # Gestión de empresas (multi-tenant)

├── **worker/** # Perfil de trabajadores

├── **workday/** # Jornadas laborales y fichaje

├── **schedule/** # Horarios y turnos

├── **timeSlot/** # Franjas horarias

├── **attendance/** # Registro de asistencia

├── **absence/** # Gestión de ausencias

├── **request/** # Sistema de solicitudes

├── **issue/** # Incidencias

├── **alarms/** # Alertas y alarmas

├── **activity/** # Catálogo de puestos

├── **site/** # Gestión de obras/sitios

├── **siteLog/** # Fichajes en obras

├── **issue-site/** # Incidencias en obras

├── **mail/** # Servicio de correo

├── **seed/** # Inicialización de datos

└── **common/** # Utilidades compartidas

Librerías y Dependencias

Dependencias de Producción

Librería	Versión	Justificación de Uso
@nestjs/core	^11.1.10	Framework principal NestJS
@nestjs/common	^11.1.10	Módulos comunes de NestJS
@nestjs/platform-express	^11.0.1	Plataforma HTTP Express
@nestjs/typeorm	^11.0.0	Integración TypeORM con NestJS
@nestjs/jwt	^11.0.2	Generación y validación de tokens JWT
@nestjs/passport	^11.0.5	Estrategias de autenticación Passport
@nestjs/config	^4.0.3	Gestión de configuración y variables de entorno
@nestjs/schedule	^6.1.1	Tareas programadas (Cron jobs)
@nestjs/throttler	^6.5.0	Rate limiting para protección contra ataques
@nestjs/swagger	^11.2.3	Documentación automática de API
@nestjs/platform-socket.io	^11.1.14	WebSockets para notificaciones en tiempo real
@nestjs/websockets	^11.1.14	Gateway de WebSockets
@nestjs/class-validator	^0.13.4	Validación de DTOs
@nestjs/class-transformer	^0.4.0	Transformación de datos

Librería	Versión	Justificación de Uso
@nestjs/mapped-types	*	Tipos mapeados para DTOs
typeorm	^0.3.28	ORM para PostgreSQL
pg	^8.16.3	Driver de PostgreSQL
bcrypt	^6.0.0	Encriptación de contraseñas
jsonwebtoken	^9.0.3	Generación de tokens JWT
passport	^0.7.0	Middleware de autenticación
passport-jwt	^4.0.1	Estrategia JWT para Passport
class-validator	^0.15.1	Validación de datos
class-transformer	^0.5.1	Transformación de objetos
cors	^2.8.5	Habilitación de CORS
cookie-parser	^1.4.7	Parseo de cookies
nodemailer	^8.0.7	Envío de correos electrónicos
date-holidays	^3.26.12	Cálculo de festivos
ioredis	^5.9.0	Cliente Redis (para sesiones/caching)
twilio	^5.11.2	Envío de SMS (opcional)
rxjs	^7.8.1	Programación reactiva

Dependencias de Desarrollo

Librería	Versión	Justificación de Uso
@nestjs/cli	^11.0.14	CLI de NestJS para scaffolding
@nestjs/testing	^11.0.1	Utilidades de testing
@nestjs/schematics	^11.0.0	Esquemas para generación de código
typescript	^5.7.3	Compilador TypeScript
ts-node	^10.9.2	Ejecución directa de TypeScript
ts-loader	^9.5.2	Loader de TypeScript para Webpack
jest	^30.0.0	Framework de testing
ts-jest	^29.2.5	Preprocesador Jest para TypeScript
eslint	^9.18.0	Linting de código
typescript-eslint	^8.20.0	Parser ESLint para TypeScript
prettier	(implicit)	Formateo de código
@types/*	Varias	Definiciones de tipos TypeScript

Entidades de la Base de Datos

Resumen de Entidades (20 entidades)

Entidad	Tabla	Descripción	Relaciones Principales
User	users	Usuarios del sistema (auth)	Company, Horario, Workday, Ausencia, Request
Company	empresas	Empresas (multi-tenant)	User, Horario, Request, Attendance
Worker	workers	Perfil laboral de trabajadores	User, Activity
Workday	jornadas	Jornadas laborales y fichajes	User, Company
Schedule	horarios	Horarios y turnos	Company, User, DistribucionDia
DistribucionDia	distribucion_dia	Distribución por día de horario	Schedule, FranjaHoraria
FranjaHoraria	franjas_horarias	Franjas horarias	DistribucionDia
Attendance	asistencias	Registro de asistencia	Company, User
Absence	ausencias	Ausencias y vacaciones	User
Request	solicitudes	Solicitudes (permisos, cambios)	User, Company, RevisionRequest
RevisionRequest	revisiones_solicitudes	Revisiones de solicitudes	Request

Entidad	Tabla	Descripción	Relaciones Principales
Issue	incidencias	Incidencias reportadas	User, Company
Alarm	alarmas	Alertas y notificaciones	User, Company
Activity	actividades	Catálogo de puestos de trabajo	User
Site	sitios	Obras o ubicaciones de trabajo	Company
SiteLog	site_logs	Fichajes en obras	Site, User
SiteLogBreak	site_log_breaks	Pausas en fichajes de obra	SiteLog
IssueSite	issue_sites	Incidencias en obras	Site, User
Mail	mails	Registro de correos enviados	-

Detalle Completo de Entidades (20 entidades)

USER (Tabla: users)

Propósito	Entidad principal de autenticación y gestión de accesos y usuarios del sistema.
Campos Clave	`id (number)` `email (text|unique)` `dni (text|unique)` `telefono (text|unique)` `password (text|select:false)` `rol (enum: SUPERADMIN, ADMIN, WORKER)` `empresaId (number)` `pin (text|select:false)` `isPinActive (boolean)` `isVerified (boolean)` `isActive (boolean)` `horarioId (number)` `actividadId (number)` `fechaInicioHorario (timestamp)` `pinUpdatedAt (timestamp)` `tokenExpiraEn (timestamp)`
Funcionalidad	Sistema de autenticación robusto integrado con JWT, control de login mediante código PIN securizado de 6 dígitos, gestión jerárquica de roles y permisos del personal, asignación automatizada de horarios laborales y actividades asignadas.
Relaciones	`Company (ManyToOne, onDelete: CASCADE)` `Horario (ManyToOne, nullable)` `Activity (ManyToOne, onDelete: SET NULL)` `Workday (OneToMany)` `Ausencia (OneToMany)` `Request (OneToMany)` `Attendance (OneToMany)` `Issue (OneToMany)` `Worker (OneToOne)`

COMPANY (Tabla: `empresas`)

Propósito	Aislamiento lógico de datos a nivel multi-empresa (Multi-tenancy), garantizando la integridad de cada cliente.
Campos Clave	`id (number)` `nombre (text|unique)` `cif (text)` `configPassword (varchar|select:false)` `direccion (text)` `codigoPostal (text)` `provincia (text)` `poblacion (text)` `contactoNombre (text)` `telefonoContacto (text)` `telefonoEmpresa (text)` `pais (text)` `email (text)` `isActive (boolean)` `codigoRegion (varchar)` `plan (varchar: trial, premium, expired)` `trialEndsAt (timestamp)` `latitud (float)` `longitud (float)` `radioPermitido (float|default:100)` `daysArchiveIssues (int|default:5)` `daysArchiveRequests (int|default:15)`
Funcionalidad	Configuración avanzada de geofencing perimetral para validación geográfica de fichajes, control del ciclo de vida y planes de suscripción de la empresa cliente, parametrización de tareas automáticas de limpieza/archivado de logs de incidencias y solicitudes, e implementación de contraseña maestra cifrada para la autogeneración dinámica de PINs de operarios.
Relaciones	`User (OneToMany, cascade: true)` `Horario (OneToMany, cascade: true, onDelete: CASCADE)` `Request (OneToMany)` `Attendance (OneToMany)`

WORKER (Tabla: `workers`)

Propósito	Almacenamiento detallado del expediente y perfil laboral y personal de los trabajadores activos.
Campos Clave	`id (number|PrimaryColumn)` `telefono (varchar|20|unique)` `dni (varchar|20|unique)` `email (varchar|150|unique)` `siteId (number)` `fechaInicioHorario (timestamp)` `nombre (varchar|100)` `apellidos (varchar|150)` `fechaNacimiento (date)` `calle (text)` `poblacion (varchar|100)` `provincia (varchar|100)` `pais (varchar|100|default:España)` `categoria (varchar|100)` `diaPertenece (int)` `isVerified (boolean)` `isActive (boolean)` `rol (enum: UserRole)` `empresaId (number)` `horarioId (number)` `actividadId (number)` `password (text|select:false)`
Funcionalidad	Mantenimiento de información de contacto, residencia e historial administrativo del empleado. Permite la asignación directa y múltiple a centros de trabajo u obras activas, vinculación estructurada de horarios/turnos y puestos sectoriales, implementando una relación de extensión estricta 1:1 con la entidad base de credenciales de usuario.
Relaciones	`Company (ManyToOne, onDelete: CASCADE)` `Horario (ManyToOne, onDelete: RESTRICT)` `Activity (ManyToOne, onDelete: SET NULL)` `User (OneToOne, onDelete: CASCADE)` `Workday (OneToMany, cascade: true)` `Ausencia (OneToMany, cascade: true)` `Site (ManyToMany)`

WORKDAY (Tabla: `jornadas`)

Propósito	Historial técnico y registro de auditoría de las jornadas laborales individuales y sus fichajes asociados.
Campos Clave	`id (number)` `inicio (timestamp)` `fin (timestamp|nullable)` `total_segundos (int|nullable)` `latitudEntrada (float|nullable)` `longitudEntrada (float|nullable)` `latitudSalida (float|nullable)` `longitudSalida (float|nullable)` `presente (boolean|default:false)` `modalidad (varchar|default:PRESENCIAL)` `tipo (varchar|default:Entrada)`
Funcionalidad	Marcación exacta de check-in y check-out con captura forzada de coordenadas geográficas en tiempo real, cálculo automatizado del intervalo exacto de tiempo efectivo trabajado (en segundos) y determinación del tipo de modalidad operativa asignada por sesión (teletrabajo/presencial).
Relaciones	`User (ManyToOne, onDelete: CASCADE)` `Company (ManyToOne)`

HORARIO (Tabla: `horarios`)

Propósito	Modelado de la matriz estructural de calendarios de trabajo, cuadrantes y turnos de la organización.
Campos Clave	`id (number)` `nombre (text)` `tipo (text)` `isActive (boolean|default:true)` `esRotativo (boolean|default:false)` `semanasRotacion (int|default:1)` `empresaId (number)` `deletedAt (timestamp|nullable)`
Funcionalidad	Soporte nativo para esquemas de cuadrantes fijos continuos o turnos rotativos complejos, parametrización de ciclos dinámicos recurrentes medidos en semanas y mecanismo de borrado lógico permanente (*soft-delete*) para preservar el histórico de fichajes pasados.
Relaciones	`Company (ManyToOne, onDelete: CASCADE)` `DistribucionDia (OneToMany, cascade: true, eager: true)` `Worker (OneToMany)`

DISTRIBUCIONDIA (Tabla: `distribucion_dia`)

Propósito	Desglose de la carga y asignación horaria obligatoria para cada día de la semana dentro de un cuadrante.
Campos Clave	`id (number)` `nombre (text|nullable)` `diaSemana (enum: LUNES..DOMINGO)` `horasDia (float|default:0)` `semanaRotacion (int|default:1)` `esLaborable (boolean|default:true)` `horarioId (number|indexed)`
Funcionalidad	Configuración de la carga horaria por jornada diaria individual, soporte explícito para variaciones semanales en cuadrantes rotativos cíclicos y diferenciación técnica obligatoria de días laborables vs fines de semana o festivos internos de empresa.
Relaciones	`Horario (ManyToOne, onDelete: CASCADE)` `FranjaHoraria (OneToMany, cascade: true, eager: true)`

FRANJAHORARIA (Tabla: `franjas_horarias`)

Propósito	Definición precisa de los tramos y sub-intervalos temporales válidos operativos de un mismo día.
Campos Clave	`id (number)` `inicio (time)` `fin (time|nullable)`
Funcionalidad	Especificación de marcas horarias límites de entrada y salida efectivas (Ej: Turnos partidos de 09:00-14:00 y 16:00-19:00), permitiendo una granularidad infinita de múltiples tramos estructurados para un único día laborable.
Relaciones	`DistribucionDia (ManyToOne, onDelete: CASCADE)`

ATTENDANCE (Tabla: attendances)

Propósito	Consolidación resumida e índices de presencia e inasistencia diaria del capital humano.
Campos Clave	`id (number)` `empresaId (int|nullable)` `usuarioId (number)` `fechaJornada (date)` `fecha (timestamp)` `presente (boolean|default:false)` `motivo (varchar|255|nullable)`
Funcionalidad	Cierre y cálculo diario consolidado de estados de asistencia, gobernado por un índice único compuesto estricto `(usuarioId, fechaJornada)` que mitiga y previene colisiones o inserciones duplicadas, con capacidad de almacenamiento de justificaciones o motivos de ausencia.
Relaciones	`User (ManyToOne, onDelete: CASCADE)` `Company (ManyToOne, onDelete: CASCADE)`

AUSENCIA (Tabla: ausencias)

Propósito	Módulo centralizado para el control operativo de bajas médicas, vacaciones anuales o licencias.
Campos Clave	`id (number)` `tipo (enum: TipoAusencia)` `fechaInicio (date)` `fechaFin (date)` `motivo (varchar|255|nullable)` `estado (enum: AusenciaEstado|default:APROBADA)` `usuarioId (int|indexed)` `empresaId (int|indexed)` `diasConsumidos (int|default:0)` `diasTotales (int|nullable)`
Funcionalidad	Tipificación estricta de ausencias (VACACIONES, ENFERMEDAD, ASUNTOS_PROPIOS), gestión de flujos de validación interna, cálculo automático de días consumidos de la bolsa anual y lógica de validación backend para evitar solapamientos temporales de fechas en el histórico.
Relaciones	`User (ManyToOne, onDelete: CASCADE)`

REQUEST (Tabla: `solicitudes`)

Propósito	Gestión del workflow transaccional e hilos de peticiones y solicitudes iniciadas por el trabajador.
Campos Clave	`id (number)` `tipo (enum: VACACIONES, ENFERMEDAD, PERMISO, ASUNTOS_PROPIOS, OTROS)` `nombre (text|nullable)` `apellidos (text|nullable)` `fechaInicio (date)` `fechaFin (date)` `motivo (text|nullable)` `estado (enum: RequestStatus|default:PENDIENTE)` `fechaSolicitud (timestamp)` `usuarioId (number)` `empresaId (number)` `updatedAt (timestamp)` `deletedAt (timestamp|nullable)`
Funcionalidad	Control completo de estados de aprobación (PENDIENTE → APROBADA / RECHAZADA), aislamiento de auditoría mediante mecanismos de *soft-delete*, registro histórico de fechas de modificación y persistencia íntegra de metadatos adjuntos para las revisiones de RRHH.
Relaciones	`User (ManyToOne, onDelete: CASCADE)` `Company (ManyToOne, onDelete: CASCADE)` `RevisionRequest (OneToOne)`

REVISIONREQUEST (Tabla: `revisiones_solicitudes`)

Propósito	Auditoría final y registro de veredictos emitidos por la capa de administración sobre las solicitudes de personal.
Campos Clave	`id (number)` `solicitudId (number)` `adminId (number)` `comentarioAdmin (text|nullable)` `fechaDecision (timestamp)` `estadoResultante (enum: RequestStatus)`
Funcionalidad	Garantiza la trazabilidad y el no-repudio al almacenar con precisión qué administrador gestionó y evaluó cada petición, junto con la persistencia cronológica de comentarios del evaluador, marcas de tiempo exactas del veredicto y mutación de estados del sistema.
Relaciones	`Request (OneToOne, onDelete: CASCADE)` `User (ManyToOne)`

ISSUE (Tabla: `incidencias`)

Propósito	Módulo central para la declaración de alertas, incidencias de fichaje u omisiones por los usuarios.
Campos Clave	`id (number)` `tipo (enum: IncidenciaTipo)` `descripcion (text)` `userId (number)` `empresaId (number)` `nombre (text|nullable)` `apellidos (text|nullable)` `estado (boolean|default:false)` `deletedAt (timestamp|nullable)` `createdAt (timestamp)` `updatedAt (timestamp)`
Funcionalidad	Tipificación estructurada de errores comunes (OLVIDO_ENTRADA, ERROR_GPS, ENTRADA_TARDIA), automatización de tareas programadas en servidor para el archivado e higienización por antigüedad, soporte integrado de *soft-delete* e índice compuesto optimizado para consultas de rendimiento (`userId, createdAt`).
Relaciones	`User (ManyToOne, onDelete: CASCADE)`

ISSUESITE (Tabla: `site_incidents`)

Propósito	Control logístico y registro de incidencias, anomalías operativas o de seguridad en proyectos u obras físicas.
Campos Clave	`id (number)` `type (enum: IncidentType|default:OTHER)` `severity (enum: IncidentSeverity|default:LOW)` `description (text)` `evidenceUrl (varchar|nullable)` `isResolved (boolean|default:false)` `resolutionNote (text|nullable)` `assignedAt (timestamp|nullable)` `resolvedAt (timestamp|nullable)` `createdAt (timestamp)`
Funcionalidad	Matriz de categorización de incidencias por tipos de riesgo y niveles de severidad crítica, soporte para almacenamiento persistente de evidencias fotográficas por URL (Cloud Object Storage), hilos de asignación a técnicos responsables y tracking temporal completo de estados de resolución.
Relaciones	`Site (ManyToOne)` `User (ManyToOne - Creador)` `User (ManyToOne - Resolutor)` `User (ManyToOne - Asignado)`

ALARM (Tabla: `alarms`)

Propósito	Motor e historial de eventos de notificaciones, alarmas automatizadas y logs en tiempo real.
Campos Clave	`id (number)` `tipo (enum: AlarmType)` `descripcion (text)` `leidaAdmin (boolean|default:false)` `fecha (date)` `leidaWorker (boolean|default:false)` `empresaId (number)` `createdAt (timestamp)`
Funcionalidad	Detección y lanzamiento desatendido de alarmas del sistema (AUSENCIA_DETECTADA, RETRASO_FICHAJE, ZONA_NO_AUTORIZADA), control independiente bicapa del estado de lectura para roles administrativos y operarios, índice restrictivo de unicidad `(usuario, tipo, fecha)` y acoplamiento con pasarela activa de WebSockets.
Relaciones	`User (ManyToOne, onDelete: CASCADE)` `Company (ManyToOne, onDelete: CASCADE)`

ACTIVITY (Tabla: `actividades`)

Propósito	Catálogo centralizado de puestos de trabajo, categorías sectoriales y roles físicos de la corporación.
Campos Clave	`id (number)` `tipo (enum: TipoPuesto|default:OTROS)` `nombre (varchar|100)` `descripcion (text|nullable)` `fecha (date)` `empresaId (number)` `activa (boolean|default:true)` `createdAt (timestamp)` `updatedAt (timestamp)`
Funcionalidad	Normalización de puestos de trabajo predefinidos por la directiva de la empresa (OPERADOR, MECANICO, ADMINISTRATIVO, etc.), soporte para descripciones funcionales extensas de tareas obligatorias y banderas de habilitación/deshabilitación para inactivar puestos del organigrama de la app.
Relaciones	`Worker (OneToMany)`

SITE (Tabla: `sites`)

Propósito	Gestión integrada de obras, localizaciones de contratas externas y centros geográficos de trabajo activos.
Campos Clave	`id (number)` `name (varchar)` `latitude (decimal|10,8)` `longitude (decimal|10,8)` `radiusMeters (int|default:100)` `direccion (text)` `poblacion (varchar|100|nullable)` `codigoPostal (text|nullable)` `provincia (text|nullable)` `pais (text|default:España)` `isActive (boolean|default:true)` `empresaId (number)` `createdAt (timestamp)` `updatedAt (timestamp)`
Funcionalidad	Representación cartográfica de precisión mediante coordenadas en formato de alta resolución decimal `(10,8)`, definición del radio de tolerancia en metros (Geofence) válido para la confirmación del fichaje del operario en obra y soporte para la asignación múltiple de trabajadores concurrentes.
Relaciones	`Company (ManyToOne)` `SiteLog (OneToMany)` `IssueSite (OneToMany)` `Worker (ManyToMany)`

SITELOGBREAK (Tabla: `site_log_breaks`)

Propósito	Registro de pausas, interrupciones, paradas técnicas o descansos dentro de una misma sesión en obra.
Campos Clave	`id (number)` `startTime (timestamp)` `endTime (timestamp|nullable)` `durationSeconds (int|nullable)`
Funcionalidad	Monitorización precisa de interrupciones reglamentarias de la actividad laboral (Ej: Comida, descansos de seguridad), realizando cálculos automáticos nativos en backend de los tiempos de pausa acumulados para su posterior descuento del total de horas efectivas facturadas.
Relaciones	`SiteLog (ManyToOne, onDelete: CASCADE)`

SITELOG (Tabla: `site_logs`)

Propósito	Libro de registro técnico y marcas de entrada/salida de operarios asociadas a centros u obras físicas.
Campos Clave	`id (number)` `siteId (number)` `userId (number)` `checkIn (timestamp)` `checkOut (timestamp|nullable)` `latitudeIn (float)` `longitudeIn (float)` `isInsideIn (boolean|default:true)` `latitudeOut (float|nullable)` `longitudeOut (float|nullable)` `isInsideOut (boolean|default:false)` `notes (text|nullable)` `totalSeconds (int|nullable)` `wasAutoClosed (boolean|default:false)` `isManual (boolean|default:false)` `editedByAdminId (number|nullable)` `serverReceivedAt (timestamp)` `createdAt (timestamp)`
Funcionalidad	Módulo transaccional de check-in/out en proyectos con doble captura de coordenadas GPS, cálculo condicional de banderas de validez perimetral (`isInsideIn`/`isInsideOut`), rutinas de cierre forzado automático para mitigar olvidos de fin de jornada, auditoría para ediciones manuales de administradores y marcas de sincronización hardware en servidor.
Relaciones	`Site (ManyToOne, onDelete: CASCADE)` `User (ManyToOne)` `SiteLogBreak (OneToMany)`

USER (Fichero: `users/entities/user.entity.ts` • Tabla: `users`)

Propósito	Gestión de usuarios, credenciales de autenticación del sistema y control directo de relaciones laborales.
Campos Específicos	`actividadId (number|nullable)` `horarioId (number|nullable)`
Campos Compartidos (Ambas entidades)	`id (number)` `email (text)` `nombre (text)` `apellidos (text)` `dni (text)` `telefono (text)` `password (text)` `rol (enum)` `isVerified (boolean)` `isActive (boolean)` `empresaId (number)` `tokenExpiraEn (timestamp)` `createdAt (timestamp)` `updatedAt (timestamp)`
Uso Operativo	Módulo centralizado de administración y gestión de usuarios, asignación dinámica de horarios de trabajo, tracking y registro consolidado de asistencias del personal y reporte/seguimiento de incidencias del sistema. **Nota técnica de exclusión:** Se elimina explícitamente cualquier relación directa con el flujo de solicitudes (Request).
Relaciones Específicas	`Worker (OneToOne)` `Attendance (OneToMany)` `Issue (OneToMany)`
Relaciones Compartidas	`Company (ManyToOne)` `Horario (ManyToOne)` `Activity (ManyToOne)` `Workday (OneToMany)` `Ausencia (OneToMany)`

> **Justificación de Arquitectura ante el Tribunal (Defensa):**
> Este diseño aplica el principio de segregación de interfaces y separación de responsabilidades (*Separation of Concerns*). La tabla `users` mantiene una separación estricta del flujo operativo de negocio de solicitudes (*Requests*), derivándolas al perfil detallado, mientras comparte el núcleo de datos del empleado y las trazas transaccionales de asistencia (`Attendance`) e históricos temporales mediante una relación 1:1 pura.

Configuraciones del Sistema

Configuración Principal (main.ts)

Propósito: Punto de entrada de la aplicación NestJS, configura el servidor globalmente.

Configuraciones implementadas:

- **Timezone**: `Europe/Madrid` - Zona horaria por defecto de la aplicación
- **Global Prefix**: `/api` - Prefijo global para todos los endpoints
- **CORS**:
  - Origen configurable via `CORS_ORIGIN` (default: `http://localhost:3567`)
  - Soporta múltiples orígenes separados por coma
  - Métodos permitidos: GET, HEAD, PUT, PATCH, POST, DELETE
  - Headers permitidos: Content-Type, Authorization
  - Credentials: true (permite cookies HTTP-only para JWT)
- **ClassSerializerInterceptor**: Transformación automática de entidades usando @Expose() y @Exclude()
- **ValidationPipe**:
  - `whitelist: true` - Elimina propiedades no definidas en DTOs
  - `forbidNonWhitelisted: true` - Lanza error si se envían propiedades no permitidas
  - `transform: true` - Transforma automáticamente tipos de datos según DTOs
- **Puerto**: Configurable via `PORT` (default: 3000)

Configuración de Módulos (app.module.ts)

Propósito: Configura todos los módulos del sistema y la conexión a base de datos.

-

Configuración TypeORM:

- **Tipo**: PostgreSQL
- **Conexión**: Variables de entorno (DB_HOST, DB_NAME, DB_USERNAME, DB_PASSWORD)
- **AutoLoadEntities**: true - Carga automáticamente todas las entidades
- **Synchronize**: true - Sincroniza esquema con entidades (desarrollo)
- **DropSchema**: false - No elimina esquema al iniciar (producción)
- **Timezone**: UTC - Consistencia global en fechas
- **Entities**: Carga todas las entidades de `/**/*.entity{.ts,.js}`

Configuración Throttler (Rate Limiting):

- **TTL**: 60000ms (1 minuto)
- **Limit**: 5 peticiones por minuto (global)
- **Propósito**: Protección contra ataques de fuerza bruta y DDoS

Configuración Schedule:

- **NestScheduleModule**: Habilita tareas programadas (Cron jobs)

Configuración ConfigModule:

- **isGlobal**: true - Disponible en todos los módulos
- **envFilePath**: `.env` - Archivo de variables de entorno

Sistema de Seguridad

Guards (Protectores de Rutas)

JwtAuthGuard

- **Propósito**: Verifica que el request tenga un token JWT válido
- **Implementación**: Extiende `AuthGuard('jwt')` de Passport
- **Funcionamiento**:
  - Extrae token del header `Authorization: Bearer <token>`
  - Valida firma y expiración del token
  - Si es válido, adjunta el usuario al request
  - Si es inválido, retorna 401 Unauthorized
- **Uso**: `@UseGuards(JwtAuthGuard)` en controladores o métodos

RolesGuard

- **Propósito**: Verifica que el usuario tenga el rol necesario para acceder al endpoint
- **Implementación**: Implementa `CanActivate`
- **Funcionamiento**:
  1. Lee metadata del decorador `@Roles()`
  2. Si no hay `@Roles`, permite acceso (solo requiere JWT)
  3. Si no hay usuario en request, lanza ForbiddenException
  4. Si el rol es SUPERADMIN, permite acceso siempre
  5. Verifica si el rol del usuario está en la lista de permitidos
  6. Si no está autorizado, lanza ForbiddenException con log de seguridad
- **Uso**: `@UseGuards(JwtAuthGuard, RolesGuard)` + `@Roles(UserRole.ADMIN, UserRole.WORKER)`

Decoradores Personalizados

@GetUser()

- **Propósito**: Extrae el usuario autenticado del request
- **Implementación**: `createParamDecorator` de NestJS
- **Funcionamiento**:
  - Accede al objeto request
  - Retorna el usuario validado por JwtStrategy
  - Opcional: puede retornar una propiedad específica (`@GetUser('id')`)
- **Uso**:

```
@Get()

@UseGuards(JwtAuthGuard)

getProfile(@GetUser() user: UserPayload) {

    return user;

}

// O propiedad específica

@Get()

@UseGuards(JwtAuthGuard)

getProfile(@GetUser('id') userId: number) {

    return userId;

}
```

@Roles()

- **Propósito**: Define qué roles pueden acceder al endpoint
- **Implementación**: `SetMetadata` de NestJS
- **Funcionamiento**: Guarda metadata que RolesGuard lee para validar
- **Uso**:

```
@Roles(UserRole.ADMIN) // Solo ADMIN

@Roles(UserRole.ADMIN, UserRole.WORKER) // ADMIN y WORKER

@Post()

@UseGuards(JwtAuthGuard, RolesGuard)

create() {}
```

Estrategias de Passport

JwtStrategy

- **Propósito**: Valida tokens JWT y extrae el payload del usuario
- **Implementación**: Extiende `PassportStrategy(Strategy)`
- **Configuración**:
  - **jwtFromRequest**: `ExtractJwt.fromAuthHeaderAsBearerToken()`
  - **ignoreExpiration**: false - No ignora expiración
  - **secretOrKey**: `JWT_SECRET` del .env
- **Método validate(payload)**:
  1. **Modo Configuración**: Si `sub === 'COMPANY_CONFIG'`, retorna payload especial para configuración
  2. **Validación Normal**:
     - Busca usuario en DB por ID
     - Si no existe, lanza UnauthorizedException
     - Si está desactivado, lanza UnauthorizedException
     - Carga relación con empresa
     - Retorna UserPayload con: id, email, rol, dni, nombre, apellido, horarioId, empresaId

Interfaces

UserPayload

- **Propósito**: Define estructura del usuario en token JWT y request
- **Campos**:
  - `id`: number - ID del usuario
  - `email`: string - Email del usuario
  - `userId`: number - ID del usuario (duplicado)
  - `nombre`: string - Nombre del usuario
  - `apellido`: string | null - Apellidos del usuario
  - `dni`: string - DNI del usuario
  - `rol`: UserRole - Rol del usuario
  - `empresaId`: number - ID de la empresa

Contraseñas

DB_USERNAME=postgres # Usuario de PostgreSQL

DB_PASSWORD=Jp10.Mnl # Contraseña de PostgreSQL

JWT_SECRET=tu_secreto_para_access # Secreto para firmar tokens de acceso

JWT_REFRESH_SECRET=tu_secreto_para_refresh # Secreto para firmar tokens de refresh

DEMO_COMPANY_PASSWORD=MiSuperPasswordDinamico2026 # Password para empresa demo seed

PIN_PEPPER=alguna_cadena_super_secreta_y_larga_1234567890!@ # "Pimienta" para cifrado HMAC-SHA256 de PINs

MAIL_USER=ac0e20001@smtp-brevo.com # Usuario SMTP (de Brevo)

MAIL_PASS=bskMpxXmumxC7KI # Contraseña SMTP (clave generada en Brevo)

SEED_SECRET_KEY=123

PGADMIN_EMAIL=admin@cticontrol.com

PGADMIN_PASSWORD=admin123

--- CONTRASEÑA PARA GENERAR ACCESO PANEL PRIVADO PIN ---

MASTER_PIN_PASSWORD=SUPERpass123.

--- PARA GENERAR PIN ---

PIN_PEPPER=alguna_cadena_super_secreta_y_larga_1234567890!@

Variables de Entorno

Archivo de Configuración del Backend (backend/.env)

Este archivo contiene todas las variables de entorno necesarias para el funcionamiento del backend en contenedores Docker.

--- CONEXIÓN A BASE DE DATOS (Interna de Docker) ---

DB_HOST=postgres_bbdd # El container_name de tu DB en Docker

DB_USERNAME=postgres # Usuario de PostgreSQL

DB_PASSWORD=Jp10.Mnl # Contraseña de PostgreSQL

DB_NAME=fichaje_cti # Nombre de la base de datos

DB_PORT=5432 # Puerto de PostgreSQL

--- CONFIGURACIÓN DE LA APP ---

NODE_ENV=development # Entorno de ejecución (development/production)

PORT=3000 # Puerto donde escucha la aplicación

CORS_ORIGIN=* # Orígenes permitidos para CORS (* = todos)

Alternativa: http://cworker1.cticontrol.com:3567,http://localhost:3567

```
TZ=Europe/Madrid            # Zona horaria de la aplicación

# --- SEGURIDAD (JWT) ---

JWT_SECRET=tu_secreto_para_access          # Secreto para firmar tokens de acceso

JWT_REFRESH_SECRET=tu_secreto_para_refresh  # Secreto para firmar tokens de refresh

JWT_ADMIN_EXPIRATION=24h                # Tiempo de expiración token admin (debe coincidir con
JWT_ADMIN_EXP_HOURS)

JWT_ADMIN_EXP_HOURS=24                  # Horas de expiración token admin (debe coincidir con
JWT_ADMIN_EXPIRATION)

JWT_WORKER_EXPIRATION=1h                # Tiempo de expiración token worker (debe coincidir con
JWT_WORKER_EXP_HOURS)

JWT_WORKER_EXP_HOURS=1                  # Horas de expiración token worker (debe coincidir con
JWT_WORKER_EXPIRATION)

JWT_EXPIRATION=1h                       # Tiempo de expiración general

JWT_REFRESH_EXPIRATION=7d               # Tiempo de expiración token de refresh (7 días)

JWT_ACCESS_EXPIRATION=1h                # Tiempo de caducidad del token de acceso

# --- LÓGICA DE NEGOCIO ---

SEED_SECRET_KEY=123                     # Clave maestra para ejecutar seed (CAMBIAR EN
PRODUCCIÓN)

MAX_FICHAR_DISTANCE=600                 # Distancia máxima en metros para fichar (geofencing)

DEFAULT_DAYS_ISSUES=5                   # Días para archivar incidencias automáticamente

DEFAULT_DAYS_REQUESTS=15                # Días para archivar solicitudes automáticamente

DEMO_COMPANY_PASSWORD=MiSuperPasswordDinamico2026  # Password para empresa demo
seed

# --- GENERACIÓN DE PIN ---

PIN_PEPPER=alguna_cadena_super_secreta_y_larga_1234567890!@  # "Pimienta" para cifrado
HMAC-SHA256 de PINs

# --- CONFIGURACIÓN DE CORREO (SMTP Brevo) ---

MAIL_HOST=smtp-relay.brevo.com          # Host del servidor SMTP
```

MAIL_PORT=587 # Puerto SMTP (587 para STARTTLS)

MAIL_USER=ac0e20001@smtp-brevo.com # Usuario SMTP (de Brevo)

MAIL_PASS=bskMpxXmumxC7Kl # Contraseña SMTP (clave generada en Brevo)

MAIL_FROM=no-reply@cticontrol.com # Correo remitente oficial

Archivo de Configuración de Base de Datos (BBDD/.env)

Este archivo contiene las variables de entorno para los contenedores de base de datos (PostgreSQL y pgAdmin).

--- CONFIGURACIÓN DE BASE DE DATOS ---

--- CONEXIÓN A BASE DE DATOS (Interna de Docker) ---

DB_HOST=postgres_bbdd # El container_name de tu DB

DB_USERNAME=postgres

DB_PASSWORD=Jp10.Mnl

DB_NAME=fichaje_cti

DB_PORT=5432

--- CONFIGURACIÓN DE LA APP ---

NODE_ENV=development

PORT=3000

CORS_ORIGIN=http://cworker1.cticontrol.com:3567,http://localhost:3567

--- SEGURIDAD (JWT) ---

JWT_SECRET=tu_secreto_para_access

JWT_REFRESH_SECRET=tu_secreto_para_refresh

JWT_ADMIN_EXPIRATION=24h #poner el mismo valor que jwt_admin_exp_hours

JWT_ADMIN_EXP_HOURS=24 #poner mismo valor que jwt_admin_expiration

JWT_WORKER_EXPIRATION=1h

```
JWT_WORKER_EXP_HOURS=1

JWT_EXPIRATION=1h

JWT_REFRESH_EXPIRATION=7d

# --- LÓGICA DE NEGOCIO ---

SEED_SECRET_KEY=123

MAX_FICHAR_DISTANCE=600

DEFAULT_DAYS_ISSUES=5

DEFAULT_DAYS_REQUESTS=15

# --- pgAdmin (opcional) ---

PGADMIN_EMAIL=admin@cticontrol.com

PGADMIN_PASSWORD=admin123

PGADMIN_PORT=5050

# --- Hosts de Base de Datos (Producción y Test) ---

DB_HOST_TEST=postgres_bbdd_test    # Container de test

DB_HOST=postgres_bbdd              # Container de producción

# --- Configuración regional ---

TZ=Europe/Madrid

# --- CONTRASEÑA PARA GENERAR ACCESO PANEL PRIVADO PIN ---

MASTER_PIN_PASSWORD=SUPERpass123.

# --- PARA GENERAR PIN ---

PIN_PEPPER=alguna_cadena_super_secreta_y_larga_1234567890!@
```

Explicación Detallada de Variables

Variables de Base de Datos

- **DB_HOST**: Nombre del contenedor Docker de PostgreSQL (`postgres_bbdd`). Es el nombre interno que Docker usa para resolver el servicio.
- **DB_HOST_TEST**: Nombre del contenedor Docker de PostgreSQL de test (`postgres_bbdd_test`).
- **DB_USERNAME**: Usuario de PostgreSQL (por defecto `postgres`).
- **DB_PASSWORD**: Contraseña del usuario de PostgreSQL. Debe coincidir con la configurada en el contenedor de base de datos.
- **DB_NAME**: Nombre de la base de datos (`fichaje_cti`).
- **DB_PORT**: Puerto de PostgreSQL (5432 es el puerto estándar).

Variables de Configuración de la App

- **NODE_ENV**: Entorno de ejecución. `development` para desarrollo local, `production` para despliegue en VPS.
- **PORT**: Puerto donde escucha la aplicación NestJS (3000).
- **CORS_ORIGIN**: Orígenes permitidos para CORS. `*` permite todos los orígenes (no recomendado en producción). Para producción, especificar dominios concretos separados por comas.
- **TZ**: Zona horaria. `Europe/Madrid` para horario español.

Variables de Seguridad JWT

- **JWT_SECRET**: Secreto usado para firmar tokens de acceso. Debe ser una cadena larga y aleatoria en producción.
- **JWT_REFRESH_SECRET**: Secreto usado para firmar tokens de refresh. Diferente al JWT_SECRET para mayor seguridad.
- **JWT_ADMIN_EXPIRATION / JWT_ADMIN_EXP_HOURS**: Tiempo de expiración de tokens para administradores (24 horas). Ambas variables deben tener el mismo valor.
- **JWT_WORKER_EXPIRATION / JWT_WORKER_EXP_HOURS**: Tiempo de expiración de tokens para trabajadores (1 hora). Ambas variables deben tener el mismo valor.
- **JWT_EXPIRATION**: Tiempo de expiración general (1 hora).
- **JWT_REFRESH_EXPIRATION**: Tiempo de expiración del token de refresh (7 días). Permite renovar el token de acceso sin re-login.
- **JWT_ACCESS_EXPIRATION**: Tiempo de caducidad del token de acceso (1 hora).

Variables de Lógica de Negocio

- **SEED_SECRET_KEY**: Clave maestra para ejecutar el seed de inicialización. Requerida en el header `x-seed-key`. CAMBIAR EN PRODUCCIÓN.
- **MAX_FICHAR_DISTANCE**: Distancia máxima en metros permitida para fichar. Si el trabajador está fuera de este radio, se genera una alarma.

- **DEFAULT_DAYS_ISSUES**: Días después de los cuales las incidencias se archivan automáticamente.
- **DEFAULT_DAYS_REQUESTS**: Días después de los cuales las solicitudes se archivan automáticamente.
- **DEMO_COMPANY_PASSWORD**: Password para la empresa demo creada por el seed.

Variables de Generación de PIN

- **PIN_PEPPER**: Cadena secreta usada como "pimienta" en el cifrado HMAC-SHA256 de los PINs. Añade una capa extra de seguridad. Debe ser larga y única.
- **MASTER_PIN_PASSWORD**: Contraseña maestra para acceder al panel privado de generación de PINs. Usada por el admin para generar PINs de trabajadores.

Variables de pgAdmin

- **PGADMIN_EMAIL**: Email de acceso a pgAdmin (`admin@cticontrol.com`).
- **PGADMIN_PASSWORD**: Contraseña de acceso a pgAdmin.
- **PGADMIN_PORT**: Puerto donde expone pgAdmin (5050).

Variables de Correo (SMTP Brevo)

- **MAIL_HOST**: Host del servidor SMTP de Brevo (`smtp-relay.brevo.com`).
- **MAIL_PORT**: Puerto SMTP (587 para STARTTLS, 465 para SSL).
- **MAIL_USER**: Usuario SMTP proporcionado por Brevo (formato: `ac0e20001@smtp-brevo.com`).
- **MAIL_PASS**: Contraseña SMTP generada en el panel de Brevo (botón "+ Generar una nueva clave...").
- **MAIL_FROM**: Correo que aparece como remitente en los emails enviados (`no-reply@cticontrol.com`). Debe ser un correo verificado en Brevo.

Configuración Docker

Justificación del Uso de Docker

¿Por qué Docker?

1. **Consistencia**: El mismo comportamiento en desarrollo y producción
2. **Aislamiento**: Cada servicio en su propio contenedor
3. **Escalabilidad**: Fácil escalado horizontal de servicios
4. **Portabilidad**: Ejecutable en cualquier sistema con Docker
5. **Versionado**: Imágenes versionadas para rollback rápido
6. **Eficiencia**: Menor consumo de recursos que máquinas virtuales

---

Proxy Nginx

Justificación del Uso de Nginx

¿Por qué Nginx como Proxy?

1. **Reverse Proxy**: Enruta solicitudes a los contenedores apropiados
2. **Balanceo de Carga**: Distribuye tráfico entre múltiples instancias
3. **SSL/TLS**: Terminación SSL para HTTPS
4. **Caching**: Mejora el rendimiento con caché estático
5. **Compresión**: Gzip para reducir tamaño de respuestas
6. **Seguridad**: Oculta puertos internos y protege contra ataques
7. **Virtual Hosts**: Múltiples dominios en un solo servidor

Capa de Acceso: Nginx Proxy (Doble Dominio)

Lanzamos el proxy inyectando la configuración para que no dependa de archivos externos en el VPS.

Nota: Los símbolos `$` están escapados como `\$` para que la terminal no los borre.

Comando de Despliegue

Eliminar contenedor anterior si existe

docker rm -f nginx_proxy

```
# Lanzar Nginx con configuración inyectada
docker run -d \
  --name nginx_proxy \
  --network red_fichaje \
  --restart always \
  -p 80:80 \
  -p 443:443 \
  nginx:1.25-alpine /bin/sh -c "echo 'events { worker_connections 1024; }
http {
    # PRODUCCIÓN
    server {
        listen 80;
        server_name fichaday.cticontrol.com;
        location /api/ {
            proxy_pass http://fichaday_backend:3000;
            proxy_set_header Host \$host;
            proxy_set_header X-Real-IP \$remote_addr;
        }
        location / {
            proxy_pass http://fichaje-frontend:3000;
        }
    }
```

```
    # TEST
    server {
        listen 80;
        server_name fichadaytest.cticontrol.com;
        location / {
            proxy_pass http://fichaday_backend_test:3000;
            proxy_set_header Host \$host;
            proxy_set_header X-Real-IP \$remote_addr;
        }
    }
}' > /etc/nginx/nginx.conf && nginx -g \"daemon off;\""
```

Conexión de Redes

```
# 1. Conectar Frontend a la red (si no estaba)
docker network connect red_fichaje fichaje-frontend || true
# 2. Conectar Proxy a la red de Test
docker network connect test_network nginx_proxy
# 3. Reiniciar para aplicar
docker restart nginx_proxy
```

Configuración Nginx Explicada

Servidor de Producción (fichaday.cticontrol.com)

```
server {
    listen 80;
    server_name fichaday.cticontrol.com;
    # API apunta al backend de prod
    location /api/ {
        proxy_pass http://fichaday_backend:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
    # La raíz apunta al frontend de prod
    location / {
        proxy_pass http://fichaje-frontend:3000;
    }
}
```

```
Servidor de Test (fichadaytest.cticontrol.com)
server {
    listen 80;
    server_name fichadaytest.cticontrol.com;
    # La raíz apunta al backend de test (sin frontend separado)
    location / {
        proxy_pass http://fichaday_backend_test:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Características de la Configuración

- **Inyección de Configuración**: La configuración se inyecta directamente en el comando `docker run`, eliminando la dependencia de archivos externos en el VPS
- **Doble Dominio**: Gestiona tanto el dominio de producción como el de test
- **Routing por Location**: `/api/` va al backend, `/` va al frontend
- **Headers**: X-Real-IP para trazabilidad de la IP original del cliente
- **Worker Connections**: 1024 conexiones simultáneas
- **Restart Policy**: `always` para asegurar que el proxy se reinicie automáticamente

---

Base de Datos PostgreSQL

Justificación del Uso de PostgreSQL

¿Por qué PostgreSQL?

1. **Relacional**: Modelo relacional robusto con integridad referencial
2. **ACID**: Garantiza atomicidad, consistencia, aislamiento y durabilidad
3. **TypeORM**: Excelente integración con TypeORM (ORM de NestJS)
4. **JSONB**: Soporte nativo de JSON para datos flexibles
5. **Performance**: Alto rendimiento con índices avanzados
6. **Escalabilidad**: Escala vertical y horizontalmente
7. **Open Source**: Sin costes de licencia
8. **Comunidad**: Amplia documentación y soporte
9. **Geolocalización**: Extensiones PostGIS para datos geoespaciales
10. **Transacciones**: Soporte completo de transacciones anidadas

Características Implementadas

- **Sincronización Automática**: `synchronize: true` en desarrollo
- **Soft Deletes**: Columnas `deleted_at` para borrado lógico
- **Índices**: Índices únicos en campos críticos (email, dni, telefono)
- **Relaciones**: CASCADE para integridad referencial
- **Transacciones**: `dataSource.transaction()` para operaciones críticas

---

pgAdmin

Propósito

pgAdmin es la herramienta de administración web oficial de PostgreSQL. Permite:

- Gestión visual de bases de datos
- Ejecución de consultas SQL
- Monitorización de rendimiento
- Gestión de usuarios y permisos
- Backup y restore de bases de datos

Acceso

- **URL**: `http://localhost:5050` (local) o `http://IP:5050` (VPS)
- **Email**: `admin@cticontrol.com`
- **Password**: `admin123` (cambiar en producción)

Conexión a Bases de Datos

1. **Producción**: Host `postgres_bbdd`, Puerto `5432`
2. **Test**: Host `postgres_bbdd_test`, Puerto `5432` (interno)

Nota de Seguridad

En producción, se recomienda no exponer pgAdmin permanentemente. Levantarlo solo cuando sea necesario para mantenimiento.

Colección Postman

Directorio de Endpoints

El proyecto incluye un directorio `EndPoints/` con colecciones de Postman para probar la API:

- **fichaje.postman_collection.json** (110 KB)

Cómo Importar en Postman

1. Abrir Postman
2. Click en "Import" (esquina superior izquierda)
3. Seleccionar "Upload Files"
4. Elegir el archivo `.json` del directorio `EndPoints/`
5. La colección se importará con todos los endpoints organizados

Script de Despliegue Automático

Archivo: deploy.ps1

Ejecutar antes en powershell: Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process

Script PowerShell para despliegue local automatizado. Requiere ejecución como administrador.

Funcionalidades del Script

1. **Verificación de Permisos**: Comprueba si se ejecuta como administrador
2. **Limpieza Radical**: Detiene contenedores y libera redes
3. **Gestión de Redes**: Crea redes Docker necesarias
4. **Liberación de Puerto 80**: Detiene servicio HTTP de Windows
5. **Levantar BBDD y pgAdmin**: Inicia contenedores de base de datos
6. **Levantar Backend**: Inicia APIs de producción y test
7. **Levantar Proxy**: Inicia Nginx con configuración local
8. **Verificación**: Muestra estado final de todos los contenedores

Ejecución del Script

En terminal como administrador

bash .\deploy.ps1

Flujo del Script

1. Verificar permisos de administrador

2. Limpiar contenedores antiguos (manteniendo volúmenes)

3. Borrar redes antiguas

4. Crear red maestra 'red_fichaje'

5. Liberar puerto 80 (net stop http)

6. Levantar BBDD y pgAdmin (cd bbdd + docker compose up -d)

7. Levantar Backend (cd backend + docker compose up -d --build)

8. Levantar Proxy (cd infra/proxy + docker compose up -d)

9. Mostrar estado final (docker ps)

Salida Esperada

--- DESPLIEGUE COMPLETADO CON ÉXITO ---

Estado actual de los servicios:

NAMES STATUS PORTS

postgres_bbdd Up 2 minutes 0.0.0.0:5432->5432/tcp
postgres_bbdd_test Up 2 minutes 0.0.0.0:5433->5432/tcp
pgadmin_bbdd Up 2 minutes 0.0.0.0:5050->80/tcp
fichaje_backend Up 1 minute 3000/tcp
fichaje_backend_test Up 1 minute 3000/tcpnginx_proxy Up 1 minute 0.0.0.0:80->80/tcp, 0.0.0.0:443->443/tcp

---

Flujo del PIN

Descripción General

El sistema de PIN permite a los trabajadores acceder rápidamente a la aplicación mediante un código de 6 dígitos, ideal para tablets en obras o puntos de fichaje.

Fase 1: Generación del PIN (Bautismo)

Actores: Admin o Encargado

Paso 1: Acceso a Zona Oculta

- El Admin accede a la zona de configuración privada
- Introduce la Contraseña Maestra (configPassword de la empresa)
- Introduce el DNI del trabajador

Paso 2: Validación Maestra

- El servidor valida que la contraseña coincida con `configPassword` de la empresa
- Busca al trabajador por DNI en la base de datos

Paso 3: Cifrado "Pimienta" (HMAC-SHA256)

1. Generar código aleatorio de 6 dígitos (ej: 456123)

2. Mezclar con PIN_PEPPER del .env

3. Generar hash HMAC-SHA256

4. Guardar hash en base de datos

Paso 4: Entrega

- El sistema muestra el PIN (456123) al Admin
- El Admin se lo entrega al trabajador (por correo, SMS o en persona)

Fase 2: Login del Trabajador (Día a Día)

Actores: Trabajador

Paso 1: Entrada de PIN

- El trabajador teclea su PIN de 6 dígitos en la tablet

- El Rate Limiter vigila (máximo 5 intentos por IP)

Paso 2: Protección contra Ataques

- Si hay más de 5 intentos fallidos, el sistema bloquea la tablet temporalmente
- Previene ataques de fuerza bruta

Paso 3: Búsqueda Silenciosa

1. Servidor recibe el PIN

2. Añade PIN_PEPPER

3. Calcula hash HMAC-SHA256

4. Busca en base de datos: "¿Quién tiene este hash?"

Paso 4: Identificación Instantánea

- Como el hash es único, la base de datos encuentra al trabajador inmediatamente
- No se expone el PIN original en ninguna consulta

Paso 5: Emisión de Tokens

- El AuthService genera un Token JWT oficial
- El trabajador accede al sistema con su rol (WORKER)

Seguridad del PIN

- **Hash HMAC-SHA256**: El PIN nunca se almacena en texto plano
- **PIN_PEPPER**: Valor secreto que añade una capa extra de seguridad
- **Rate Limiting**: Protección contra ataques de fuerza bruta
- **Único por Trabajador**: Cada trabajador tiene su propio PIN
- **Regeneración**: El Admin puede regenerar el PIN en cualquier momento

---

Envío de Correo del PIN

Servicio de Correo (MailService)

El sistema utiliza **Nodemailer** para el envío de correos electrónicos con el PIN a los trabajadores.

Función: sendWorkerPin

Parámetros:

- `email`: Correo del trabajador
- `nombre`: Nombre del trabajador
- `pin`: PIN de 6 dígitos
- `esNuevoPin`: Boolean (si es regeneración o alta nueva)

Asunto del Correo:

- Nuevo PIN: " Tu nuevo PIN de seguridad - Fichaje CTI"
- Alta nueva: " Bienvenido - Tu PIN de acceso a Fichaje CTI"

Adjunto del Logo

El correo incluye el logo de la empresa como adjunto incrustado (CID):

Proveedor de Correo

El proyecto utiliza **Brevo** (antes Sendinblue) como proveedor SMTP:

- **Panel de Logs**: https://app.brevo.com/transactional/email/logs
- **Ventajas**: API gratuita hasta 300 emails/día, dashboard de tracking
- **Configuración**: SMTP en puerto 587 con STARTTLS

Seed de Inicialización

Propósito del Seed

El módulo **Seed** permite inicializar el sistema con datos base, creando el usuario SuperAdmin necesario para empezar a gestionar empresas.

Servicio: SeedsService

Requiere: Header `x-seed-key` con el valor de `SEED_SECRET_KEY` del .env
}

Endpoint del Seed

POST /api/seeds/run

Headers: x-seed-key: 123

Response: { ok: true, message: 'Sistema inicializado (SuperAdmin).' }

Flujo de Inicialización Proyecto

1. Ejecutar seed con clave maestra

2. Crear SuperAdmin (superadmin@cti.com)

3. Login como SuperAdmin

4. Crear primera empresa

5. Crear Admin de la empresa

6. Login como Admin

7. Crear horarios

8. Dar de alta trabajadores

9. Asignar horarios a trabajadores

10. Operativa norma

Módulos del Sistema

1. Auth (Autenticación)

Funcionalidad: Sistema de autenticación y autorización JWT

Características:

- Login con email/DNI y password
- Generación de tokens JWT (access + refresh)
- Refresh token para renovación automática
- 3 roles: SUPERADMIN, ADMIN, WORKER
- Guards: JwtAuthGuard, RolesGuard
- Decoradores: @GetUser(), @Roles()

Endpoints:

- `POST /api/auth/login` - Inicio de sesión
- `POST /api/auth/logout` - Cierre de sesión
- `GET /api/auth/me` - Información del usuario autenticado
- `POST /api/auth/refresh` - Renovar token

Utilidad: Base de seguridad del sistema, controla quién puede acceder y qué puede hacer.

---

2. Users (Usuarios)

Funcionalidad: Gestión de usuarios del sistema

Características:

- CRUD completo de usuarios
- Validación de email, DNI, teléfono (únicos)
- Encriptación de passwords con bcrypt
- Soft delete (borrado lógico)
- Filtrado por empresa (multi-tenancy)

Endpoints:

- `POST /api/users/crear-user` - Crear usuario
- `GET /api/users/list-users` - Listar usuarios

- `GET /api/users/find/:id` - Buscar usuario
- `PUT /api/users/update/:id` - Actualizar usuario
- `DELETE /api/users/delete/:id` - Eliminar usuario

Utilidad: Gestión centralizada de todos los usuarios del sistema.

---

3. Company (Empresas)

Funcionalidad: Gestión multi-tenant de empresas

Características:

- Creación de empresas con geolocalización
- Configuración de radio permitido para fichajes
- Configuración de días de archivo (solicitudes, incidencias)
- Password maestra por empresa (configPassword)
- Planes: trial, premium, expired

Endpoints:

- `POST /api/empresa/crear-empresa` - Crear empresa + Admin
- `GET /api/empresa/list-empresas` - Listar empresas
- `GET /api/empresa/find/:id` - Detalle de empresa
- `PUT /api/empresa/update/:id` - Actualizar empresa
- `DELETE /api/empresa/delete/:id` - Eliminar empresa
- `PUT /api/empresa/settings/archive` - Configurar archivo

Utilidad: Núcleo del multi-tenancy, aísla datos por empresa.

---

4. Worker (Trabajadores)

Funcionalidad: Perfil laboral detallado de trabajadores

Características:

- Información personal y laboral
- Asignación de horarios
- Asignación de actividades/puestos
- Reasignación masiva de horarios
- Sincronización con tabla users

Endpoints:

- `POST /api/worker/crear-worker` - Crear trabajador
- `GET /api/worker/list-workers` - Listar trabajadores
- `GET /api/worker/find/:id` - Ficha de trabajador
- `PUT /api/worker/update/:id` - Actualizar trabajador
- `DELETE /api/worker/delete/:id` - Eliminar trabajador
- `PUT /api/worker/assign-schedule/:id` - Asignar horario
- `POST /api/worker/reasignar-masivo` - Reasignación masiva

Utilidad: Gestión del perfil laboral de los trabajadores.

---

5. Workday (Jornadas)

Funcionalidad: Registro de jornadas laborales y fichaje

Características:

- Check-in con geolocalización
- Check-out con cálculo de tiempo trabajado
- Validación de radio permitido (geofencing)
- Detección de retrasos
- Historial de jornadas
- Dashboard en tiempo real

Endpoints:

- `POST /api/jornada/check-in` - Iniciar jornada
- `PUT /api/jornada/check-out` - Finalizar jornada
- `GET /api/jornada/status` - Estado de jornada activa
- `GET /api/jornada/historial` - Historial personal
- `GET /api/jornada/rango` - Filtrar por fechas
- `GET /api/jornada/dashboard/resumen` - Dashboard empresa
- `GET /api/jornada/dashboard/stats` - Estadísticas trabajador

Utilidad: Core del sistema de fichaje, registro de presencia.

---

6. Schedule (Horarios)

Funcionalidad: Gestión de horarios y turnos

Características:

- Horarios fijos y rotativos
- Ciclos de rotación configurables
- Distribución por día de la semana
- Franjas horarias anidadas
- Cálculo de horas objetivo vs real
- Simulación de rotación

Endpoints:

- `POST /api/schedule/crear-schedule` - Crear horario
- `GET /api/schedule/list-schedules` - Listar horarios
- `GET /api/schedule/find/:id` - Detalle de horario
- `DELETE /api/schedule/delete/:id` - Eliminar horario
- `PUT /api/schedule/editar/:id` - Actualizar horario
- `GET /api/schedule/stats/objetivo` - Horas objetivo
- `GET /api/schedule/stats/real` - Horas trabajadas
- `GET /api/schedule/stats/progreso` - % cumplimiento
- `GET /api/schedule/test-rotacion` - Simular rotación
- `GET /api/schedule/turno-hoy` - Turno de hoy

Utilidad: Planificación de turnos, base para cálculo de horas.

---

7. TimeSlot (Franjas Horarias)

Funcionalidad: Gestión de franjas horarias

Características:

- Creación de franjas horarias
- Validación de solapamientos
- Vinculación a horarios
- Auditoría de cambios

Endpoints:

- `POST /api/franja-horaria/crear` - Crear franja
- `GET /api/franja-horaria/list-franja-horaria` - Listar franjas
- `GET /api/franja-horaria/find/:id` - Buscar franja
- `PUT /api/franja-horaria/update/:id` - Actualizar franja

- `DELETE /api/franja-horaria/delete/:id` - Eliminar franja
- `GET /api/franja-horaria/by-schedule/:id` - Franjas de horario

Utilidad: Componentes básicos de los horarios.

---

8. Attendance (Asistencia)

Funcionalidad: Registro simple de asistencia

Características:

- Registro de entrada/salida simple
- Motivo de ausencia
- Historial personal y global
- Filtrado por trabajador

Endpoints:

- `POST /api/asistencia/crear-asistencia` - Registrar asistencia
- `GET /api/asistencia/my-history` - Historial personal
- `GET /api/asistencia/empresa` - Historial global
- `GET /api/asistencia/user/:id` - Historial trabajador
- `GET /api/asistencia/list/:id` - Fichajes trabajador

Utilidad: Registro manual de asistencia complementario al fichaje.

---

9. Absence (Ausencias)

Funcionalidad: Gestión de ausencias y vacaciones

Características:

- Tipos: VACACIONES, ENFERMEDAD, PERSONAL
- Estados: PENDIENTE, APROBADA, RECHAZADA
- Balance de días disponibles (22 por ley)
- Validación anti-solapamiento
- Cálculo de festivos con date-holidays

Endpoints:

- `POST /api/absences` - Solicitar ausencia
- `GET /api/absences/my-absences` - Mis ausencias
- `GET /api/absences/pending` - Pendientes (Admin)
- `GET /api/leaves/range` - Crear rango de vacaciones
- `GET /api/leaves/my-leaves` - Mis permisos
- `GET /api/leaves/user/:id` - Permisos trabajador

Utilidad: Gestión de vacaciones y permisos.

---

10. Request (Solicitudes)

Funcionalidad: Sistema de solicitudes con workflow

Características:

- Tipos de solicitudes (permisos, cambios)
- Workflow: Worker solicita → Admin revisa
- Estados: PENDIENTE, APROBADA, RECHAZADA
- Revisiones con notas
- Historial completo
- Archivo automático por antigüedad

Endpoints:

- `POST /api/request/crear-request` - Crear solicitud
- `PUT /api/request/check/:id` - Aprobar/rechazar
- `GET /api/request/company-request` - Solicitudes empresa
- `GET /api/request/find/:id` - Detalle solicitud
- `PATCH /api/request/edit-request/:id` - Modificar pendiente
- `GET /api/request/historial` - Historial por estado
- `GET /api/request/my-request` - Mis solicitudes
- `GET /api/request/user/:userId` - Solicitudes trabajador

Utilidad: Workflow de aprobación para cambios y permisos.

---

11. Issue (Incidencias)

Funcionalidad: Gestión de incidencias

Características:

- Reporte de incidencias
- Asignación a trabajadores
- Resolución con notas
- Archivo automático (configurable por empresa)
- Limpieza con cron job

Endpoints:

- `POST /api/issues` - Crear incidencia
- `GET /api/issues` - Listar incidencias
- `PUT /api/issues/:id` - Actualizar incidencia
- `DELETE /api/issues/:id` - Eliminar incidencia

Utilidad: Gestión de problemas y excepciones.

---

12. Alarms (Alarmas)

Funcionalidad: Sistema de alertas en tiempo real

Características:

- Tipos: UNAUTHORIZED_ZONE, LATE_CHECKIN, etc.
- WebSocket para notificaciones en tiempo real
- Anti-spam (prevención de duplicados)
- Estados: pendiente, revisada, resuelta
- Notas de gestión

Endpoints:

- `GET /api/alarmas/pendientes` - Alarmas pendientes
- `PUT /api/alarmas/revisar/:id` - Marcar como leída
- `PUT /api/alarmas/resolver/:id` - Resolver con nota
- `GET /api/alarmas/user/:userId` - Alarmas trabajador

Utilidad: Alertas automáticas para supervisión.

---

13. Activity (Actividades)

Funcionalidad: Catálogo de puestos de trabajo

Características:

- Creación de puestos (ej: Ingeniero, Operario)
- Asignación a trabajadores
- Listado de puestos activos
- Conteo de trabajadores por puesto

Endpoints:

- `POST /api/actividades/crear` - Crear puesto
- `PUT /api/actividades/editar/:id` - Modificar puesto
- `DELETE /api/actividades/delete/:id` - Eliminar puesto
- `GET /api/actividades/catalogo` - Catálogo de puestos
- `GET /api/actividades/:id` - Detalle de puesto
- `GET /api/actividades/user/:id` - Conteo trabajadores
- `GET /api/actividades/list/:id` - Trabajadores asignados

Utilidad: Clasificación de trabajadores por puesto.

---

14. Site (Obras/Sitios)

Funcionalidad: Gestión de obras o ubicaciones de trabajo

Características:

- Creación de obras con geolocalización
- Asignación a empresas
- Gestión de trabajadores en obra

Endpoints:

- `POST /api/sites` - Crear obra
- `GET /api/sites` - Listar obras
- `GET /api/sites/:id` - Detalle de obra
- `PUT /api/sites/:id` - Actualizar obra
- `DELETE /api/sites/:id` - Eliminar obra

Utilidad: Gestión de múltiples ubicaciones de trabajo.

---

15. SiteLog (Fichajes en Obras)

Funcionalidad: Fichaje específico para obras

Características:

- Check-in/out en obras
- Validación GPS de la obra
- Pausas (breaks)
- Historial de fichajes
- Reporte de horas para nómina

Endpoints:

- `POST /api/site-logs/check-in` - Entrada en obra
- `PUT /api/site-logs/check-out/:id` - Salida de obra
- `POST /api/site-logs/start-break/:id` - Iniciar pausa
- `POST /api/site-logs/end-break/:id` - Finalizar pausa
- `GET /api/site-logs/my-history` - Mi historial
- `GET /api/site-logs/active/:siteId` - Trabajadores activos
- `PUT /api/site-logs/admin-update/:id` - Corrección admin
- `GET /api/site-logs/report/:userId` - Reporte horas

Utilidad: Fichaje geolocalizado en obras específicas.

---

16. IssueSite (Incidencias en Obras)

Funcionalidad: Incidencias específicas de obras

Características:

- Reporte de problemas en obra
- Asignación a operarios
- Gestión de tareas asignadas
- Resolución con notas

Endpoints:

- `POST /api/site-issues` - Crear incidencia obra
- `GET /api/site-issues/site/:siteId` - Incidencias obra
- `PUT /api/site-issues/assign/:id` - Asignar operario
- `GET /api/site-issues/my-tasks` - Mis tareas
- `PUT /api/site-issues/resolve/:id` - Resolver incidencia

Utilidad: Gestión de problemas en obras específicas.

17. Mail (Correo)

Funcionalidad: Servicio de envío de correos

Características:

- Envío de PIN a trabajadores
- Plantillas HTML personalizadas
- Adjuntos (logo)
- Integración con SMTP (Brevo)
- Manejo de errores

Endpoints:

- `POST /api/mail/send-pin` - Enviar PIN
- `GET /api/mail` - Listar correos
- `POST /api/mail` - Crear correo
- `PUT /api/mail/:id` - Actualizar correo
- `DELETE /api/mail/:id` - Eliminar correo

Utilidad: Comunicación con trabajadores (PIN, notificaciones).

---

18. Seed (Inicialización)

Funcionalidad: Inicialización del sistema

Características:

- Creación de SuperAdmin
- Generación masiva de trabajadores demo
- Limpieza de usuarios demo
- Validación con clave maestra

Endpoints:

- `POST /api/seeds/run` - Ejecutar seed
- `POST /api/seeds/bulk-workers` - Generar trabajadores masivos
- `DELETE /api/seeds/clear-demo` - Limpiar demo

Utilidad: Punto de partida del sistema, datos iniciales.

---

19. WorkerAuth (Autenticación PIN)

Funcionalidad: Login rápido con PIN

Características:

- Login con PIN de 6 dígitos
- Rate limiting (5 intentos)
- Generación de PIN con hash HMAC-SHA256
- Regeneración de PIN
- Envío de PIN por correo

Endpoints:

- `POST /api/worker-auth/company-config-login` - Login config
- `POST /api/worker-auth/activate-pin` - Activar PIN
- `POST /api/worker-auth/login-pin` - Login con PIN
- `PUT /api/worker-auth/regenerate-pin` - Regenerar PIN

Utilidad: Acceso rápido para trabajadores en tablets.

---

20. Common (Utilidades)

Funcionalidad: Utilidades compartidas

Características:

- Decoradores personalizados
- Guards personalizados
- DTOs base
- Interfaces comunes
- Constantes globales

Utilidad: Reutilización de código en toda la aplicación.

Flujo Completo de la Aplicación

Fase 1: Inicialización del Sistema

1. Ejecutar script deploy.ps1 (levanta infraestructura)

2. Ejecutar seed: POST /api/seeds/run (con x-seed-key)

3. Se crea SuperAdmin: superadmin@cti.com

Fase 2: Configuración de Empresa

1. Login SuperAdmin: POST /api/auth/login

2. Crear empresa: POST /api/empresa/crear-empresa

   - Nombre, CIF, geolocalización

   - Se crea automáticamente el Admin de la empresa

3. Login Admin de la empresa

Fase 3: Planificación

1. Crear horarios: POST /api/schedule/crear-schedule

   - Definir turnos, ciclos de rotación

   - Añadir distribuciones por día

   - Añadir franjas horarias

2. Crear actividades/puestos: POST /api/actividades/crear

Fase 4: Gestión de Personal

1. Dar de alta trabajadores: POST /api/worker/crear-worker

   - Datos personales, DNI, email, teléfono

   - Se crea usuario en tabla users y perfil en workers

2. Asignar horario: PUT /api/worker/assign-schedule/:id

3. Asignar actividad: PUT /api/worker/update/:id

4. Generar PIN: POST /api/worker-auth/regenerate-pin

   - Se envía por correo automáticamente

Fase 5: Operativa Diaria (Trabajador)

Opción A: Login con PIN (Tablet)

1. POST /api/worker-auth/login-pin

2. POST /api/jornada/check-in

   - Validación GPS

   - Detección de retraso

   - Creación de alarma si fuera de zona

3. POST /api/jornada/check-out

   - Cálculo de minutos trabajados

   - Validación de salida anticipada

Opción B: Login con Email (Web/Móvil)

1. POST /api/auth/login (email + password)

2. POST /api/jornada/check-in

3. POST /api/jornada/check-out

Fase 6: Gestión de Solicitudes

1. Trabajador crea solicitud: POST /api/request/crear-request

   - Permiso, cambio de horario, etc.

2. Admin recibe notificación (WebSocket)

3. Admin revisa: PUT /api/request/check/:id

   - Aprobar o rechazar

- Añadir nota de revisión

4. Trabajador recibe notificación del resultado

Fase 7: Gestión de Ausencias

1. Trabajador solicita ausencia: POST /api/absences

   - Tipo: VACACIONES, ENFERMEDAD, PERSONAL

   - Fechas

2. Sistema valida:

   - Días disponibles (balance)

   - Solapamiento con otras ausencias

   - Festivos

3. Admin aprueba/rechaza

4. Sistema actualiza balance

Fase 8: Supervisión y Alarmas

1. Sistema genera alarmas automáticamente:

   - Fichaje fuera de zona (UNAUTHORIZED_ZONE)

   - Retraso en check-in (LATE_CHECKIN)

   - Salida anticipada (EARLY_CHECKOUT)

2. Admin ve alarmas pendientes: GET /api/alarmas/pendientes

3. Admin revisa: PUT /api/alarmas/revisar/:id

4. Admin resuelve: PUT /api/alarmas/resolver/:id

   - Añade nota de gestión

Fase 9: Reportes y Estadísticas

1. Admin consulta dashboard: GET /api/jornada/dashboard/resumen

   - Trabajadores presentes hoy

- Retrasos

- Ausencias

2. Admin consulta estadísticas: GET /api/jornada/dashboard/stats

- Horas trabajadas

- % cumplimiento horario

3. Trabajador consulta su progreso: GET /api/schedule/stats/progreso

Fase 10: Mantenimiento Automático

Cron jobs (ejecución diaria a medianoche):

1. Archivar solicitudes antiguas (configurable por empresa)

2. Archivar incidencias antiguas (configurable por empresa)

3. Limpieza de alarmas resueltas

Guía de Despliegue

Despliegue Local (Desarrollo)

Requisitos Previos

- Docker Desktop instalado
- PowerShell con permisos de administrador
- Git (para clonar el repositorio)

Pasos de Despliegue

1. Clonar Repositorio

git clone [url-del-repo]

cd Fichaje_Cti

yarn install

2. Ejecutar Script Automático

En PowerShell como administrador

bash .\deploy.ps1

El script ejecuta automáticamente:

- Limpieza de contenedores antiguos
- Creación de redes Docker
- Liberación de puerto 80
- Levantamiento de BBDD y pgAdmin
- Levantamiento de Backend (prod + test)
- Levantamiento de Proxy Nginx

3. Verificar Estado

docker ps

Deberías ver 6 contenedores:

- postgres_bbdd
- postgres_bbdd_test
- pgadmin_bbdd
- fichaje_backend
- fichaje_backend_test
- nginx_proxy

4. Inicializar Sistema

```
# Ejecutar seed

curl -X POST http://localhost:3000/api/seeds/run \

  -H "x-seed-key: 123"
```

5. Acceder a Servicios

- **API Producción**: http://localhost:3000/api
- **API Test**: http://localhost:3000/api (en red test)
- **pgAdmin**: http://localhost:5050
- **Frontend (vía Nginx)**: http://localhost

Despliegue en Producción (VPS)

Requisitos Previos

- VPS con Docker instalado
- Acceso SSH al servidor
- Cuenta en Docker Hub
- Dominios configurados (fichaday.cticontrol.com, fichadaytest.cticontrol.com)

Pasos de Despliegue

1. Preparación Local (Build de Imágenes)

```
# Login en Docker Hub

docker login

# Construir imagen Backend

cd backend
```

```
docker build -t javierm735/fichaday-backend:latest .

docker push javierm735/fichaday-backend:latest

# Construir imagen BBDD (si es personalizada)

cd BBDD

docker build -t javierm735/postgres_bbdd_0.10:latest .

docker push javierm735/postgres_bbdd_0.10:latest

# Construir imagen Nginx

cd infra/proxy

docker build -t javierm735/nginx_proxy-0.03:latest .

docker push javierm735/nginx_proxy-0.03:latest
```

2. Preparación del Servidor

```
# Conectar por SSH al VPS

ssh usuario@tu-vps-ip

# Crear redes Docker

docker network create red_fichaje

docker network create test_network

# Crear volúmenes

docker volume create postgres_data_bbdd

docker volume create postgres_data_test
```

3. Levantar Base de Datos

```
# Base de Datos Producción

docker run -d \

  --name postgres_bbdd \

  --network red_fichaje \
```

```
  --restart always \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_PASSWORD=Jp10.Mnl \
  -e POSTGRES_DB=fichaje_cti \
  -v postgres_data_bbdd:/var/lib/postgresql/data \
  -p 5432:5432 \
  javierm735/postgres_bbdd_0.10:latest
# Base de Datos Test
docker run -d \
  --name postgres_bbdd_test \
  --network test_network \
  --restart always \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_PASSWORD=Jp10.Mnl \
  -e POSTGRES_DB=fichaje_cti \
  -v postgres_data_test:/var/lib/postgresql/data \
  -p 5433:5432 \
  javierm735/postgres_bbdd_0.10:latest
```

4. Levantar Backend

Con el archivo env en la carpeta env

```
# Backend Producción
docker run -d \
  --name fichaday_backend \
  --network red_fichaje \
```

```
  --restart always \
   -e DB_HOST=postgres_bbdd \
  -e TYPEORM_SYNCHRONIZE=true \
  --env-file env/.env \
  javierm735/fichaday-backend-0.32:latest
# Backend Test
docker run -d \
  --name fichaday_backend_test \
  --network test_network \
  --restart always \
  -e DB_HOST=postgres_bbdd_test \
  -e TYPEORM_SYNCHRONIZE=true \
  --env-file env/.env \
  javierm735/fichaday-backend-0.30:latest
```

5. Levantar Nginx Proxy

```
# Descargar imagen
docker pull javierm735/nginx_proxy-0.03:latest
# Levantar proxy
docker run -d \
  --name nginx_proxy \
  --network red_fichaje \
  --restart always \
  -p 80:80 \
  -p 443:443 \
```

```
  javierm735/nginx_proxy-0.03:latest

-----------------

  docker run -d \

  --name nginx_proxy \

  --network red_fichaje \

  --restart always \

  -p 80:80 \

  -p 443:443 \

  nginx:1.25-alpine /bin/sh -c "echo 'events { worker_connections 1024; }

http {

    # PRODUCCIÓN

    server {

        listen 80;

        server_name fichaday.cticontrol.com;

        location /api/ {

            proxy_pass http://fichaday_backend:3000;

            proxy_set_header Host \$host;

            proxy_set_header X-Real-IP \$remote_addr;

        }

        location / {

            proxy_pass http://fichaje-frontend:3000;

        }

    }

    # TEST
```

```
    server {
        listen 80;
        server_name fichadaytest.cticontrol.com;
        location / {
            proxy_pass http://fichaday_backend_test:3000;
            proxy_set_header Host \$host;
            proxy_set_header X-Real-IP \$remote_addr;
        }
    }
}' > /etc/nginx/nginx.conf && nginx -g \"daemon off;\""

# Conectar a red de test
docker network connect test_network nginx_proxy

# Reiniciar proxy
docker restart nginx_proxy
```

6. Conectar Frontend (si existe)

```
# Conectar frontend de producción
docker network connect red_fichaje fichaje-frontend-prod

# Conectar frontend de test
docker network connect test_network fichaje-frontend-test
```

7. Verificar Despliegue

```
# Ver contenedores
docker ps

# Ver logs de backend
```

```
docker logs fichaday_backend

# Ver logs de nginx

docker logs nginx_proxy
```

8. Inicializar Sistema

```
# Ejecutar seed (desde el VPS o local)

curl -X POST http://fichaday.cticontrol.com/api/seeds/run \

  -H "x-seed-key: 123"
```

Resumen de Comandos del README

Instalación Local

```
# Clonar y entrar

git clone [url-del-repo]

cd Fichaje_Cti

yarn install

# Ejecutar script de despliegue

bash .\deploy.ps1
```

Levantar Infraestructura Manualmente

```
# Crear redes

docker network create red_fichaje

docker network create test_network

# Levantar BBDD
```

```
cd BBDD

docker compose --env-file .env up -d

# Levantar Backend

cd ../backend

docker compose --env-file .env up -d --build

# Levantar Proxy

cd ../infra/proxy

docker compose up -d
```

Despliegue Producción (VPS)

```
# Crear redes y volúmenes

docker network create red_fichaje

docker network create test_network

docker volume create postgres_data_bbdd


docker volume create postgres_data_test
```

Conclusión

Este documento proporciona una visión completa y técnica del sistema Fichaday CTI, cubriendo todos los aspectos necesarios para entender, desplegar y mantener la aplicación.

Puntos Clave:

- **Arquitectura Enterprise**: NestJS + TypeORM + PostgreSQL
- **Multi-tenancy**: Aislamiento completo por empresa
- **Seguridad**: JWT + Roles + PIN con hash HMAC-SHA256
- **Geolocalización**: Validación GPS para fichajes
- **Docker**: Contenedores para consistencia y escalabilidad
- **Nginx**: Reverse proxy para enrutamiento y seguridad
- **Automatización**: Script de despliegue y cron jobs
- **Documentación**: Colección Postman para pruebas

El sistema está listo para producción con una arquitectura escalable, segura y mantenible.

Documento generado el: 27 de Mayo de 2026
Versión del sistema: v0.7
Autor: F. Javier Méndez Terrones